Algorithmic Trading Data Architecture: REST vs. WebSockets

In automated trading, it is crucial to use the right communication protocol for the right job. We typically separate data requirements into two main categories: static/transactional data and dynamic/real-time data.

Feature	REST API (Static/Transactional)	WebSocket API (Dynamic/Real-Time)
Purpose	Information retrieval, configuration, order placement, order modification, and historical data fetching.	Continuous, low-latency delivery of live market updates.
Data Flow	Synchronous Request ➡️ Response	Asynchronous Push (Broker sends data to bot continuously)
Latency	Higher (Seconds/Milliseconds)	Ultra-Low (Milliseconds)
Best For	1. Market Status (Holidays, Open/Close). 2. **Pre-trade Setup** (Fetching instrument keys, expiries). 3. **Placing/ Modifying Orders**. 4. Backtesting (Historical Candlestick data).	1. **Live Price Feed** (LTP, Bid/Ask, Depth). 2. **Strategy Triggering** (Input for strategy logic). 3. **Order Execution Status** (Updates on placed orders).
Example	`get_next_expiry()`, `is_market_open_today()`, `place_order()`.	Receiving a new price tick for `NIFTY26SEP2420500CE`.

1. The REST API: The Setup and Control Layer

The REST (Representational State Transfer) API is used for single actions where you send a request and wait for a response. In your `TradingScheduler`, you are already using the Upstox REST API effectively for:

1. **Initialization:** Checking market holidays (`is_market_open_today`).
2. **Preparation:** Finding the current Nifty Spot, listing all expiries, and retrieving the specific instrument keys for the ATM CE and PE contracts (`get_atm_instrument_keys`).
3. **Trade Management (Future):** When we write the strategy logic, the REST API will be used to send commands to the broker to **place, modify, or cancel trades**.

2. The WebSocket API: The Real-Time Feed

The WebSocket API is required for the actual trading logic. Unlike REST, a WebSocket connection is

persistent: once established, the broker continuously **pushes** data to your bot without the bot having to ask for it repeatedly.

In your architecture:

- The `prepare_session` function uses **REST** to get the two specific instrument keys (CE and PE).
- The `start_streaming` function then instantiates your `DataStreamer` class, which connects the **WebSocket** and tells the broker: "Send me all real-time tick data for *these two keys*."
- The strategy logic inside your `DataStreamer` will then execute its fast-paced decisions based on this immediate, low-latency stream of ticks.

Conclusion on Data Usage

Since your strategy focuses on short windows of high volatility (9:15-9:40 AM and 3:00-3:15 PM):

- **Live Bot:** Needs only the WebSocket feed for real-time prices during those specific times.
- **Analysis/Backtesting:** Needs the REST API to download historical candlestick data (e.g., 5-minute candles for the last 5 days) to help tune the strategy *before* the market opens.

The next logical step, as you suggested, is to discuss how to handle and store that **historical data for analysis** so you can tune your bot's parameters effectively!